PH-Google Backend — Complete Development Workflow

From Backend Foundation (Step 1) to the Planned Search Engine Architecture

Purpose: This document gives the complete backend roadmap: what has already been built, what is currently being built, and what we plan to build next.

1. Overall Backend Architecture

The backend is being built as a complete search-engine pipeline rather than only a crawler. The crawler discovers web pages, MongoDB maintains durable crawl/source data, Redis and BullMQ coordinate jobs, OpenSearch provides retrieval and ranking, and the Search API exposes ranked results to the frontend.

```
Seed / Crawl Configuration → Crawl Job Manager → BullMQ + Redis → Web Crawler → robots.txt + Sitemap + Link Discovery → Durable MongoDB Frontier → URL Validation → Page Fetching → Text Processing → MongoDB Source Storage + OpenSearch Index → Query Parser → Search/Ranking → Search API → PH-Google Frontend
```

2. Step 1 — Backend Foundation

The first stage establishes the backend application, configuration, database connectivity, API structure, crawler services, and search infrastructure required by later phases.

• **Backend application:** Node.js backend structure, configuration and service organization.

• **MongoDB:** persistent storage for pages, crawler state/frontier and related data.

• **OpenSearch:** search index used for fast retrieval and relevance scoring.

• **Redis + BullMQ:** background job orchestration for crawler work.

• **Environment/configuration:** crawler limits, OpenSearch, Redis, MongoDB and application settings.

• **Docker:** reproducible backend, worker, OpenSearch, Redis and dashboard services.

3. Step 2 — Web Crawler Core

The crawler becomes the data-ingestion engine of PH-Google.

• Seed URLs and topic-based crawl configuration.

• robots.txt checking and crawl-delay handling.

• URL normalization and duplicate prevention.

• Allowed-domain/subdomain and exclusion rules.

• Page fetching with timeout, size and content-type controls.

• Link extraction and discovery.

• Per-host request control and crawler politeness.

• Retry, lease and failure handling.

4. Step 3 — Durable Crawl Frontier

MongoDB is used as the durable URL frontier so discovered URLs survive worker restarts and can be scheduled, claimed and retried safely.

• URL canonicalKey and deduplication.

• pending / fetching / done / failed / skipped states.

• Atomic URL claiming.

• Leases and orphan recovery.

• Priority and depth handling.

• Retry/backoff behavior.

• Scheduled re-crawling.

• Separation of re-crawls from old crawl-job budgets.

5. Step 4 — Queue and Worker Architecture

Redis/BullMQ distributes crawler work while preserving per-host control.

```
Seed/Crawl Job → BullMQ → Host Slice → Worker → Frontier Claim → Fetch → Process → Index → Continue/Finish
```

• Host-based queue processing.

• Worker concurrency controls.

• Maintenance scheduling.

• Job status and failure handling.

• Orphan-job recovery.

• Controlled re-enqueueing when frontier work remains.

6. Step 5 — Page Processing and Indexing

Fetched pages are cleaned and converted into searchable documents.

• Extract title, headings, description, keywords, URL and page content.

• Process and normalize text.

• Store source/page information in MongoDB.

• Index searchable fields in OpenSearch.

• Keep OpenSearch content bounded to 60,000 characters.

• Maintain content hashes and metadata needed for deduplication and reindexing.

7. Step 6 — Search Engine and Ranking

The search backend converts user queries into OpenSearch queries and ranks matching documents.

• Question/query parsing.

• Exact title and phrase matching.

• BM25 relevance scoring.

• Fuzzy matching for suitable terms.

• Field weighting for title, headings, description, keywords, URL and content.

• Quality multiplier.

• Freshness multiplier.

• Site/domain matching signals.

• Pagination and duplicate handling.

• Search result highlighting/snippet generation.

8. Step 6A — Continuous Crawl (COMPLETED)

Continuous Crawl was added so a crawl can continue through controlled continuation jobs when its individual page budget is reached.

• Disabled by default through configuration.

• Continuation limit prevents uncontrolled chains.

• Parent frontier work is safely handed to continuation jobs.

• Atomic continuation creation prevents duplicates.

• Closed-job race conditions were fixed.

• Scheduled re-crawls are detached from old crawl jobs.

• The continuation chain remains subject to crawl budgets.

Verification: 23 suites, 767 tests passed, 0 failed, 0 skipped. Backend and worker deployment was verified.

9. Step 6B — Sitemap Discovery (CURRENT ROADMAP)

Sitemap discovery will allow the crawler to find URLs from robots.txt Sitemap entries, sitemap.xml files and sitemap indexes instead of relying only on hyperlinks.

```
robots.txt → Sitemap URL → sitemap.xml / Sitemap Index → Extract → Validate → Deduplicate → MongoDB Frontier → Existing Crawler
```

• Master switch off by default.

• Maximum sitemap files per discovery run.

• Maximum sitemap index depth.

• Maximum URLs read per file.

• Maximum URLs added to the frontier.

• Maximum sitemap response size.

• Loop/cycle protection.

• Same-host sitemap fetching and cross-host redirect protection.

• XML/XXE and gzip-bomb protections.

• Reuse existing crawl delay and timeout controls.

10. Step 6B Phase A — Sitemap Parser & Fetcher

Next implementation: build the standalone sitemap parser and safe sitemap fetcher before wiring them into the crawler.

• crawler/sitemapParser.js

• crawler/sitemapFetcher.js

• CRAWLER_SITEMAPS and CRAWLER_SITEMAP_* configuration.

• Unit tests for XML parsing, gzip, size limits, malformed XML, redirect safety and timeout/error handling.

• No live crawl and no production data changes during this phase.

11. Step 6B — Remaining Implementation Phases

Phase B — Sitemap Discovery Service

Build the service that coordinates robots Sitemap entries, sitemap files/indexes, validation, deduplication and insertion into the existing frontier.

Phase C — Crawler Integration

Connect sitemap discovery to the worker after the seed page is indexed. Keep the feature opt-in and preserve existing crawl behavior when disabled.

Phase D — Continuous Crawl + Seed Catalog

Carry sitemap configuration into continuation jobs, support per-seed opt-in and ensure sitemap discovery occurs only once per crawl chain/host.

Phase E — Verification

Run linting, formatting, full test suite and one small controlled live crawl using a sitemap-enabled seed and a small page budget.

12. Crawl Hygiene Improvements

After sitemap discovery, the crawler roadmap includes additional hygiene improvements to make crawling more correct and respectful.

• Canonical URL handling.

• meta robots and X-Robots-Tag handling.

• Conditional HTTP requests where appropriate.

• Improved robots.txt cache behavior and size limits.

• Better handling of crawl errors and unsupported resources.

13. Crawl Source Diversification

The current corpus is heavily influenced by Wikipedia seeds. The next crawler growth stage should diversify sources so the search engine is not dependent on one site or one content type.

• Add carefully selected domains/topics.

• Use controlled allowed-domain scopes.

• Keep per-topic crawl budgets.

• Avoid uncontrolled broad crawling.

• Measure corpus quality and coverage before increasing crawl size.

14. Controlled 10K+ Crawl Stage

Once sitemap discovery and crawl hygiene are stable, the system can move from the current small corpus toward a controlled 10,000+ page corpus.

• Increase crawl volume gradually.

• Monitor MongoDB storage and frontier growth.

• Monitor OpenSearch heap and indexing performance.

• Measure crawl throughput and failure rates.

• Measure search quality against representative queries.

• Tune worker concurrency and per-host politeness.

15. Search Quality Improvements

After the crawler has a broader corpus, ranking can be improved using additional signals.

• Authority/popularity signals.

• Link-graph relationships.

• Better near-duplicate detection.

• Improved snippets and highlighting.

• More robust query understanding.

• Broader language/analyzer support.

• Potential geographic relevance where appropriate.

• A judged query set for measurable ranking evaluation.

16. Large-Scale Architecture — Later Stage

These components should only be introduced when the corpus and workload justify them.

• Redis Bloom/seen filters for very large URL volumes.

• Object storage for large raw page content.

• Distributed crawler/fetcher workers.

• Multi-node OpenSearch and multiple shards/replicas.

• Large-scale near-duplicate detection.

• Link graph storage and processing.

• Potential use of Common Crawl or web-scale datasets for later expansion.

17. Complete Development Roadmap

Stage	Main Work	Status
Step 1	Backend foundation, DB, OpenSearch, Redis/BullMQ, configuration	Completed
Step 2	Crawler core, robots, fetching, link discovery	Completed
Step 3	Durable MongoDB frontier and crawl state	Completed
Step 4	Queue/worker architecture and maintenance	Completed
Step 5	Page processing, indexing and content optimization	Completed
Step 6	Search, query parsing, ranking and pagination	Completed / improving
Step 6A	Continuous Crawl + race-condition fixes	Completed
6B Phase A	Sitemap parser + fetcher	Next
6B Phase B	Sitemap discovery service	Planned
6B Phase C	Worker integration	Planned
6B Phase D	Continuous Crawl + seed integration	Planned
6B Phase E	Full verification + controlled live crawl	Planned
Next	Crawler hygiene + source diversification	Planned

Stage	Main Work	Status
Next	Controlled 10K+ crawl	Planned
Next	Search quality/ranking improvements	Planned
Later	100K+/1M+ scale architecture	Future

18. Final Target Workflow

Discovery: Seeds + robots.txt + sitemap + links → **Frontier:** MongoDB → **Orchestration:** Redis/BullMQ → **Crawling:** controlled host workers → **Processing:** extract/clean/normalize → **Storage:** MongoDB → **Index:** OpenSearch → **Query:** parser + retrieval + ranking → **API:** search endpoint → **Frontend:** PH-Google results and AI answer experience.

The immediate next milestone is Step 6B Phase A, followed by the remaining sitemap phases. The architecture is intentionally incremental: every major crawler capability is tested before it is connected to production crawling.